

Technical Documentation

orthodox.net Flask Backend

Generated Documentation

March 15, 2026

Abstract

This document is a comprehensive technical reference for the orthodox.net Flask Backend — a Python/Flask REST API that aggregates content from Ghost CMS, YouTube, and Google Calendar into a unified JSON interface for the St. Nicholas Orthodox Church website. The application is deployed on Render.com with staging and production environments, and uses Supabase (PostgreSQL) as a caching layer to reduce external API quota usage. This reference is intended for senior/full-stack developers and DevOps engineers who need to understand, maintain, deploy, or extend the application. It covers system architecture with dependency diagrams, complete API endpoint documentation with request/response examples, database schemas for both cache tables, caching strategy with TTL configuration and error fallback behavior, core module internals, Render.com deployment configuration with service IDs and URLs, environment variable and credential management, error handling patterns, logging infrastructure, security considerations, known technical debt, a maintenance checklist for project continuity, and the full project history organized by development milestone.

Contents

1	Executive Summary	5
2	Introduction	6
2.1	Source Repository	6
2.2	What This Application Does	6
2.3	Architecture Goals	6
3	System Architecture	7
3.1	High-Level Overview	7
3.2	Core Technology Stack	7
3.3	Module Dependency Graph	8
3.4	Project Structure	10

4	Database Layer	10
4.1	Database Schema	10
4.1.1	Calendar Events Table (<code>db_schema.sql</code>)	11
4.1.2	YouTube Videos Table (<code>video_cache_schema.sql</code>)	11
4.1.3	Additional Tables in <code>db_schema.sql</code> (Future Use)	12
5	Caching Layer	12
5.1	Cache Strategy	12
5.2	Calendar Events Cache (<code>calendar_cache.py</code>)	12
5.3	YouTube Videos Cache (<code>video_cache.py</code>)	14
5.4	Supabase Client Initialization	14
6	API Endpoints — Complete Reference	15
6.1	System Endpoints	15
6.2	Article Endpoints (Ghost CMS)	15
6.2.1	GET <code>/onet_articles</code>	15
6.2.2	GET <code>/onet_article_data</code>	15
6.2.3	GET <code>/onet_article?article_id={id}</code>	15
6.2.4	GET <code>/article_by_slug?slug={slug}</code>	16
6.2.5	GET <code>/onet_articles_to_list</code>	16
6.2.6	GET <code>/parish_life</code>	16
6.2.7	GET <code>/motd</code>	16
6.2.8	GET <code>/latest_article?article_id={id}</code>	16
6.2.9	GET <code>/legacy_article?filename={filename}</code>	16
6.2.10	GET <code>/test_article</code>	17
6.3	Video Endpoints (YouTube)	17
6.3.1	GET <code>/latest_videos</code>	17
6.3.2	GET <code>/homilies_videos</code>	17
6.3.3	GET <code>/childrens_videos</code>	17
6.3.4	GET <code>/catechism_videos</code>	17
6.3.5	GET <code>/view_video_cache?video_type={type}</code>	18
6.3.6	POST <code>/cache_videos?video_type={type}&max_results={n}</code>	18
6.4	Calendar Endpoint	18
6.4.1	GET <code>/schedule</code>	18
7	Core Module Details	18

7.1	Ghost CMS Integration (<code>fetch_ghost.py</code>)	18
7.2	YouTube Integration (<code>fetch_youtube.py</code>)	19
7.3	Calendar Integration (<code>fetch_calendar.py</code>)	20
7.4	Legacy Article Scraper (<code>fetch_legacy_article.py</code>)	20
8	Error Handling Patterns	20
8.1	General Pattern	20
8.2	Cache Failure Resilience	20
8.3	External API Errors	21
9	Logging	21
10	Environment Configuration	21
11	Deployment & DevOps	22
11.1	Hosting Platform	22
11.2	Render Environments	24
11.2.1	Production Environment	24
11.2.2	Staging Environment	24
11.3	Render Service Configuration	24
11.4	Deployment Workflow	25
11.5	Procfile	25
11.6	CORS Configuration	25
11.7	Swagger / HTTPS Detection	25
12	Development Setup	26
12.1	Quick Start	26
12.2	Running Tests	26
12.3	Test Organization	26
12.4	Compiling This Document	27
13	Security Considerations	27
14	Known Technical Debt	27
15	Maintenance Checklist	28
A	Project History by Milestone	30

A.1	Phase 1: Initial Setup and Core API (Sep 2023)	30
A.2	Phase 2: YouTube Playlists and Ghost CMS (Oct–Nov 2023)	30
A.3	Phase 3: Article Features and Metadata (Jan–Jul 2024)	30
A.4	Phase 4: Testing, Database, and Infrastructure (Dec 2024)	31
A.5	Phase 5: Content Endpoints and Calendar (Jan–Feb 2025)	31
A.6	Phase 6: Migration to Render and Supabase Caching (Aug 2025)	32
A.7	Phase 7: Stabilization and Bug Fixes (Dec 2025)	32
B	Document Revision History	33

1 Executive Summary

orthodox.net is the website of St. Nicholas Orthodox Church in McKinney, Texas. The orthodox.net Flask Backend is a Python/Flask REST API that serves as the data layer for the site, aggregating content from Ghost CMS, YouTube, and Google Calendar into a unified JSON interface consumed by the frontend. The project began in September 2023 and has evolved through seven development phases, from an initial Heroku deployment to the current Render.com infrastructure with Supabase caching.

Architecture. The backend is a single Flask application (`app.py`) deployed on Render.com with Gunicorn as the WSGI server. External integrations are organized into separate modules: `fetch_ghost.py` for articles, `fetch_youtube.py` for videos, and `fetch_calendar.py` for church events. Each cached integration has a dedicated cache module (`calendar_cache.py`, `video_cache.py`) backed by Supabase (PostgreSQL). Calendar events are cached for 24 hours and YouTube videos for 6 hours, significantly reducing external API quota consumption. The API is public with no authentication; CORS is open to all origins. Interactive Swagger documentation is auto-generated via Flasgger and available at the `/docs` endpoint.

API Surface. The backend exposes endpoints across three content domains: articles from Ghost CMS (`/onet_articles`, `/onet_article`, `/article_by_slug`, `/parish_life`, `/motd`, and others), videos from YouTube (`/latest_videos` with caching, plus playlist endpoints for homilies, children’s sermons, and catechism), and calendar events from Google Calendar (`/schedule` with caching). Admin endpoints (`POST /cache_videos`, `/view_video_cache`) allow manual cache management. A legacy article scraper fetches historical content from the original **orthodox.net** site by extracting HTML between comment markers.

Deployment. The application runs on two Render.com Web Services (Starter plan, Virginia region): `production-orthodox-net-flask` (deployed from the `main` branch) and `staging-orthodox-net-flask` (deployed from `staging`). Both auto-deploy on push to their respective branches. Environment variables for Supabase and Google Calendar are configured per-service in the Render dashboard. Ghost CMS and YouTube credentials are currently hardcoded in source files rather than read from environment variables — migrating these is a known technical debt item.

Technology Stack. Python 3.12.3, Flask 2.1.2, Werkzeug 2.0.3, Gunicorn; Supabase (PostgreSQL) for caching; `requests` for HTTP calls to Ghost CMS and YouTube; `google-api-python-client` for Calendar; `python-dotenv` for environment management; `beautifulsoup4` for HTML parsing; `pytz` for US/Central timezone handling. Testing uses `pytest` with `pytest-mock` and `pytest-cov`.

This Document. This technical reference covers system architecture, all API endpoints with request/response examples, database schemas, caching strategies, module internals, deployment configuration, error handling, logging, security considerations, and known technical debt. Section 15 provides a maintenance checklist; Appendix A provides the project history by milestone; Appendix B records this document’s revision history.

Repository: <https://github.com/GZancewicz/onet-flask-backend>
Production: <https://production-orthodox-net-flask.onrender.com>
Staging: <https://staging-orthodox-net-flask.onrender.com>

2 Introduction

The orthodox.net Flask Backend is a REST API that aggregates content from multiple external services and presents a unified interface to frontend clients. It powers the orthodox.net platform for St. Nicholas Orthodox Church.

2.1 Source Repository

The source code is hosted on GitHub at:

<https://github.com/GZancewicz/onet-flask-backend>

The `main` branch is deployed to production and `staging` to the staging environment (both on Render.com). All pull requests should target `main` unless specifically intended for staging testing. For a narrative overview of the project’s development history, see Appendix A.

2.2 What This Application Does

1. **Article management** — Proxies articles from a Ghost CMS instance, with tag-based filtering, slug lookups, and heading reformatting for frontend consumption.
2. **Video aggregation** — Fetches and caches YouTube videos from the church’s channel and curated playlists (homilies, children’s sermons, catechism).
3. **Calendar integration** — Retrieves upcoming church events from Google Calendar, formats them with human-readable dates/times, and caches results.
4. **Legacy content** — Scrapes historical articles from the legacy `orthodox.net` website.

2.3 Architecture Goals

- Single deployment artifact (one Flask app, one Gunicorn process)
- Minimize external API calls through Supabase caching (YouTube and Google Calendar have strict quotas)
- CORS-open so any frontend can consume the API
- Self-documenting via Swagger UI at `/docs`
- Comprehensive logging for debugging production issues

3 System Architecture

3.1 High-Level Overview

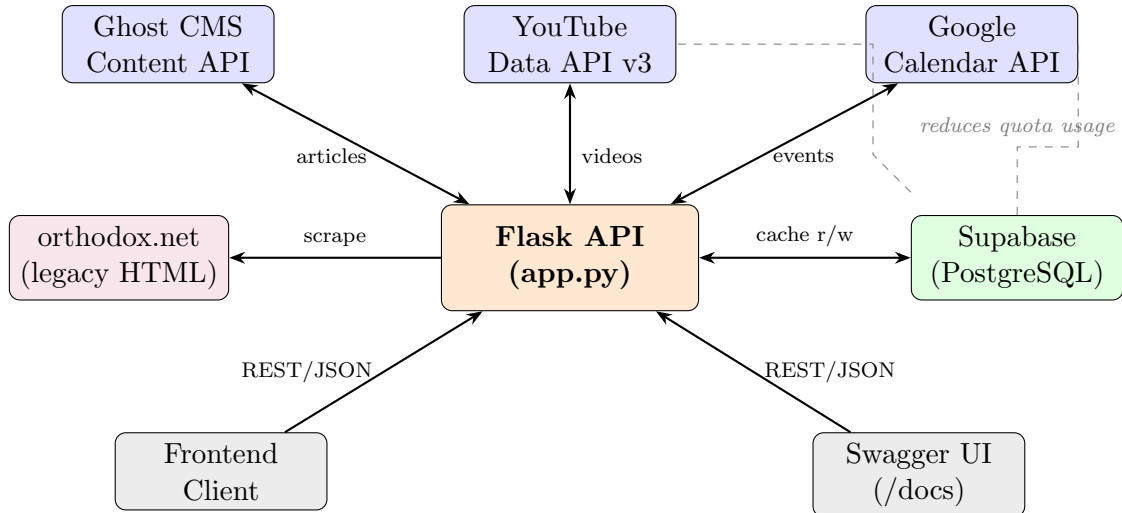


Figure 1: System architecture. The Flask API is the central hub. Supabase caches YouTube and Calendar data to stay within API quotas. Ghost CMS and legacy site are called directly on every request (no caching).

3.2 Core Technology Stack

Component	Version	Purpose
Flask	2.1.2	Web framework
Werkzeug	2.0.3	WSGI utilities (pinned for Flask 2.1 compat)
Flasgger	0.9.7.1	Swagger UI auto-generation
flask_cors	4.0.0	CORS middleware
Gunicorn	(latest)	Production WSGI server
requests	2.28.2	HTTP client for Ghost, YouTube, legacy site
google-api-python-client	2.91.0	Google Calendar API client
supabase	2.18.0	Supabase Python client
SQLAlchemy	2.0.23	Database ORM (used in db/ setup scripts)
psycopg2-binary	2.9.10	PostgreSQL adapter
pytz	2023.3	Timezone conversions (US/Central)
python-dotenv	1.0.1	.env file loading
beautifulsoup4	4.12.3	HTML parsing
pytest	7.4.3	Test framework
pytest-mock	3.11.1	Mock fixtures for tests
pytest-cov	4.1.0	Coverage reporting
Python	3.12.3	Runtime (specified in runtime.txt)

3.3 Module Dependency Graph

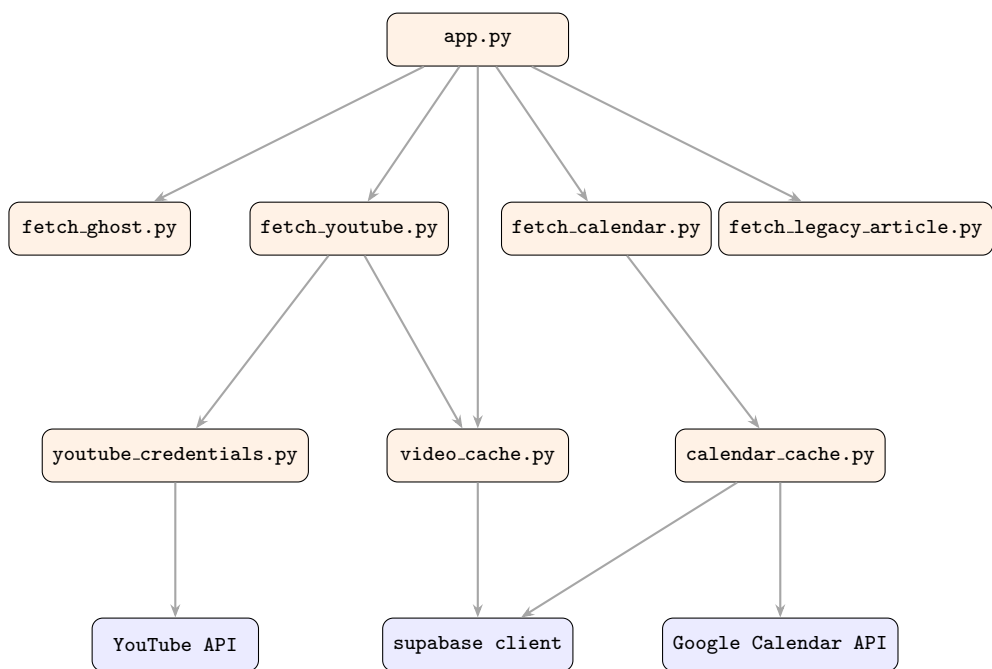


Figure 2: Module dependency graph. Each fetch module has its own cache module backed by Supabase.

3.4 Project Structure

```
onet-flask-backend/
|-- app.py                # Main Flask app, all REST endpoints
|-- fetch_ghost.py        # Ghost CMS Content API client
|-- fetch_youtube.py      # YouTube Data API v3 client + caching
|-- youtube_credentials.py # YouTube API key (hardcoded)
|-- fetch_calendar.py     # Google Calendar API client + caching
|-- calendar_cache.py     # Supabase ops for calendar_events table
|-- video_cache.py        # Supabase ops for youtube_videos table
|-- fetch_legacy_article.py # Scrapes orthodox.net legacy HTML
|-- conftest.py           # Pytest configuration
|-- db/
|   |-- src/
|   |   |-- setup_db.py      # DB initialization via SQLAlchemy
|   |   +-- setup_youtube_tables.py # YouTube table creation
|   +-- test/                # DB setup tests
|-- modules/
|   +-- youtube/
|       |-- src/fetch_youtube.py # Alternate YouTube module
|       +-- test/                # Unit + live integration tests
|-- test/                    # Main test suite
|-- legacy_test/             # Legacy tests (Ghost, YouTube)
|-- data/bulletin/          # Bulletin PDF extraction utilities
|-- util/onet-v2-utils/     # Misc utility scripts
|-- db_schema.sql           # Calendar + YouTube table schemas
|-- video_cache_schema.sql  # Video cache table (simplified schema)
|-- requirements.txt        # Python dependencies (used by Render)
|-- runtime.txt             # Python version for Render (3.12.3)
|-- Procfile                # Gunicorn start command (legacy Heroku
|                           # format, still used by Render)
|-- .env.sample             # Environment variable template
|-- compile_doc.sh          # Compiles this LaTeX document
+-- CLAUDE.md               # AI assistant development guide
```

Note on Procfile: The Procfile was originally created for Heroku. Render.com also reads this file format, so it remains functional. It contains a single line: `web: gunicorn app:app`. If migrating to a different hosting platform, this file may need to be replaced with the platform’s equivalent (e.g., a Dockerfile, `render.yaml`, etc.).

4 Database Layer

The application uses Supabase (hosted PostgreSQL) exclusively as a caching layer. There is no user data, authentication, or transactional data stored.

4.1 Database Schema

Two SQL files define the schema. Both should be run in the Supabase SQL editor during initial setup.

4.1.1 Calendar Events Table (db_schema.sql)

```
1 CREATE TABLE IF NOT EXISTS calendar_events (
2     id                SERIAL PRIMARY KEY,
3     event_id          VARCHAR(255) UNIQUE NOT NULL,
4     summary           TEXT,
5     start_time        TIMESTAMP WITH TIME ZONE,
6     end_time          TIMESTAMP WITH TIME ZONE,
7     location          TEXT,
8     description       TEXT,
9     raw_data          JSONB,
10    date_key           DATE NOT NULL,
11    cached_at         TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
12    UNIQUE(event_id, date_key)
13 );
14
15 CREATE INDEX idx_calendar_events_date
16     ON calendar_events(date_key);
17 CREATE INDEX idx_calendar_events_cached
18     ON calendar_events(cached_at);
```

Key design decisions:

- `event_id` is the Google Calendar event ID; upserts use `ON CONFLICT event_id`.
- `date_key` enables fast date-range queries without parsing `start_time`.
- `raw_data` stores the complete Google Calendar API response as JSONB, so the application can re-extract any field without a schema migration.
- `cached_at` is compared against `CACHE_TTL_HOURS` (default 24) to determine staleness.

4.1.2 YouTube Videos Table (video_cache_schema.sql)

```
1 CREATE TABLE IF NOT EXISTS youtube_videos (
2     video_id          TEXT PRIMARY KEY,
3     title             TEXT,
4     description       TEXT,
5     thumbnail_url     TEXT,
6     published_at      TIMESTAMP,
7     channel_id        TEXT,
8     playlist_id       TEXT,
9     video_type        TEXT,    -- 'latest','homilies','childrens','catechism'
10
11     raw_data          JSONB,
12     cached_at         TIMESTAMP DEFAULT NOW()
13 );
14
15 CREATE INDEX IF NOT EXISTS idx_youtube_videos_type
16     ON youtube_videos(video_type);
17 CREATE INDEX IF NOT EXISTS idx_youtube_videos_cached_at
18     ON youtube_videos(cached_at);
19 CREATE INDEX IF NOT EXISTS idx_youtube_videos_playlist
20     ON youtube_videos(playlist_id);
```

A PostgreSQL function for periodic cleanup is also defined:

```
1 CREATE OR REPLACE FUNCTION clean_old_video_cache()
2 RETURNS void AS $$
3 BEGIN
4     DELETE FROM youtube_videos
5     WHERE cached_at < NOW() - INTERVAL '7 days';
6 END;
7 $$ LANGUAGE plpgsql;
```

4.1.3 Additional Tables in db_schema.sql (Future Use)

The db_schema.sql file also defines two additional tables that are provisioned but **not yet used** by the application code:

- youtube_videos — A more detailed YouTube table with playlist_ids TEXT[], tags TEXT[], metadata JSONB, and updated_at. This overlaps with video_cache_schema.sql; the simpler schema is what video_cache.py actually uses.
- youtube_transcriptions — Planned table for storing video transcriptions with full-text search (GIN index on to_tsvector). Foreign-keyed to the YouTube videos table.

5 Caching Layer

5.1 Cache Strategy

5.2 Calendar Events Cache (calendar_cache.py)

Property	Value
Table	calendar_events
TTL	24 hours (configurable: CACHE.TTL_HOURS)
Upsert key	event_id
Query pattern	Filter by date_key range + cached_at
Fallback (API error)	Returns expired cache if available, else []
Fallback (no API key)	Returns expired cache if available
Cleanup	clear_old_cache(days_to_keep=7)

Key functions in calendar_cache.py:

- get_cached_events(date_key) — Single-date lookup with TTL check.

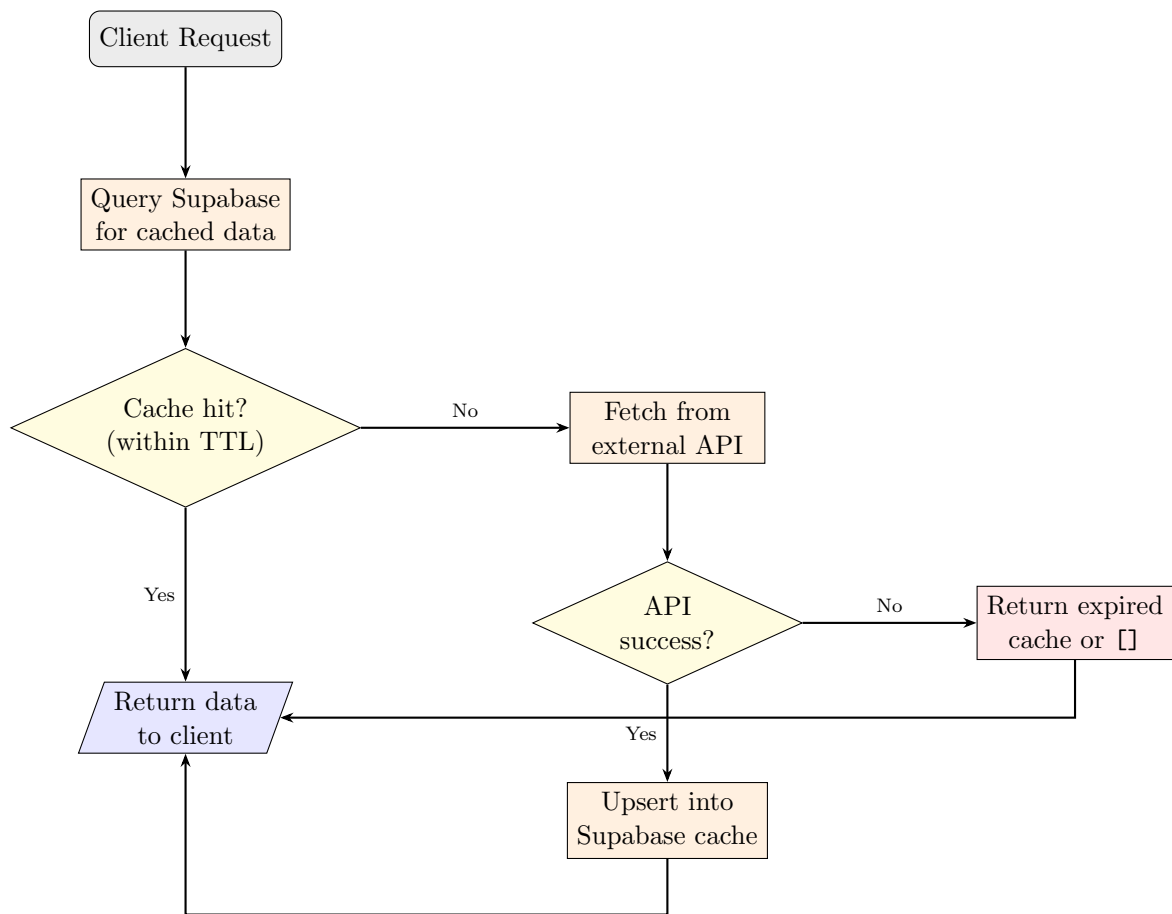


Figure 3: Cache lookup flow with error fallback. On API failure, the calendar module returns expired cache data; the video module returns an empty list.

- `get_events_for_date_range(start_date, end_date)` — Range query; returns raw_data JSONB.
- `cache_events(events_json)` — Iterates over Google Calendar events, extracts `date_key` from each, upserts.
- `clear_old_cache(days_to_keep=7)` — Deletes entries older than N days.

5.3 YouTube Videos Cache (`video_cache.py`)

Property	Value
Table	<code>youtube_videos</code>
TTL	6 hours (<code>VIDEO_CACHE.TTL_HOURS</code> constant)
Upsert conflict key	<code>video_id</code>
Query pattern	Filter by <code>video_type</code> + <code>cached_at >= (now - TTL)</code>
Applies to	<code>/latest_videos</code> endpoint only
Does NOT apply to	<code>/homilies_videos</code> , <code>/childrens_videos</code> , <code>/catechism_videos</code>
Cleanup function	<code>clear_old_video_cache(days_to_keep=7)</code>

Important: Only the `/latest_videos` endpoint uses the cache automatically. The playlist endpoints (`/homilies_videos`, etc.) hit the YouTube API directly on every request. The admin endpoint `POST /cache_videos` can be used to manually populate the cache for any type.

Key functions in `video_cache.py`:

- `get_cached_videos(video_type, playlist_id)` — TTL-aware lookup; returns raw_data.
- `cache_videos(videos_data, video_type, playlist_id)` — Upserts each video item.
- `clear_cache_by_type(video_type)` — Deletes all entries for a given type (used by `POST /cache_videos`).
- `get_all_cached_videos(video_type)` — Admin query ignoring TTL; returns metadata only.

5.4 Supabase Client Initialization

Both `calendar_cache.py` and `video_cache.py` initialize their Supabase clients identically at module load time:

```

1 supabase_url = os.getenv('SUPABASE_URL')
2 supabase_key = os.getenv('SUPABASE_KEY')
3 if supabase_url and supabase_key:
4     supabase = create_client(supabase_url, supabase_key)
5 else:
6     supabase = None # Caching disabled gracefully

```

If credentials are missing, caching is silently disabled — the application continues to function by calling external APIs directly on every request. All cache functions check if not supabase: return None/False/[] at the top.

6 API Endpoints — Complete Reference

6.1 System Endpoints

Method	Path	Description
GET	/	HTML welcome page
GET	/heartbeat	Health check; returns {"status":"ok"}
GET	/docs	Interactive Swagger UI (Flasgger)
GET	/apispec.json	Raw OpenAPI spec JSON

6.2 Article Endpoints (Ghost CMS)

All article endpoints call the Ghost CMS Content API at:
<https://orthodox-net.ghost.io/ghost/api/content/>

6.2.1 GET /onet_articles

Returns the full Ghost CMS response for all posts tagged `orthodox_net`, including the `posts` array with title, HTML content, tags, and authors. No pagination on the Flask side (Ghost's default page size applies).

6.2.2 GET /onet_article_data

Returns article metadata **without** full HTML content. Paginates through all Ghost pages (100 per page) to collect every post. Response is an array of:

```
{
  "id": "652a8f969a71080001718f5b",
  "title": "Article Title",
  "excerpt": "Custom excerpt text...",
  "posted_at": "2024-01-15T12:00:00.000Z",
  "tags": ["orthodox_net", "parish_life"],
  "authors": ["Author Name"],
  "url": "https://orthodox-net.ghost.io/article-slug/"
}
```

Note: The `excerpt` field maps to Ghost's `custom_excerpt`, not the auto-generated excerpt. Articles without a custom excerpt will have `null` here.

6.2.3 GET /onet_article?article_id={id}

Fetches a single article by Ghost CMS ID. Returns:

```
{
  "title": "Article Title",
  "content": "<strong>Reformatted HTML...</strong>",
  "featured_image": "https://images.unsplash.com/..."
}
```

Heading reformatting: All `<h1>` through `<h6>` tags are replaced with `<strong>` tags. This is done in `fetch_ghost.py:reformat_headings()` to give the frontend full control over heading styles.

6.2.4 GET /article_by_slug?slug={slug}

SEO-friendly article lookup. Returns the same structure as `/onet_article` plus additional fields: `slug`, `id`, `tags` (raw Ghost tag objects), `authors` (raw Ghost author objects), `published_at`.

6.2.5 GET /onet_articles_to_list

Filtered article list. Fetches all `orthodox.net` posts, then filters client-side for articles whose `primary_tag.name` starts with `onet_`. Returns array of `{id, primary_tag_slug, title, excerpt}`.

6.2.6 GET /parish_life

Fetches all article metadata via `fetch_tagged_post_data("orthodox.net")`, filters for articles having both `orthodox.net` and `parish_life` tags, sorts by `posted_at`, then fetches the full article content for the most recent one. Response matches `/onet_article`.

6.2.7 GET /motd

Same logic as `/parish_life` but filters for the `motd` tag instead.

6.2.8 GET /latest_article?article_id={id}

Fetches the article list (calls `get_article_list()` internally, result unused) then fetches the article by ID. Functionally equivalent to `/onet_article`.

6.2.9 GET /legacy_article?filename={filename}

Scrapes a legacy article from <https://www.orthodox.net/articles/{filename}.html>. Extracts content between `<!--BEGIN-CONTENT-->` and `<!--END-CONTENT-->` comment markers. Returns `{"body": "..."} or an error if markers are not found.`

6.2.10 GET /test_article

Debug endpoint. Fetches article with hardcoded ID 652a8f969a71080001718f5b. Not intended for production use.

6.3 Video Endpoints (YouTube)

YouTube integration uses channel ID UCb_MxJmD_J4peQtRHj-qBKA and the YouTube Data API v3. API key is loaded from `youtube_credentials.py`.

Common video response format (all video endpoints return an array of these):

```
{
  "title": "Video Title",
  "description": "Video description...",
  "url": "https://www.youtube.com/watch?v=VIDEO_ID",
  "thumbnail": "https://i.ytimg.com/vi/.../hqdefault.jpg",
  "published_at": "2025-01-15T12:00:00Z",
  "channel_title": "Orthodox Net",
  "duration": "PT15M30S",
  "view_count": "1234",
  "like_count": "56",
  "dislike_count": "N/A"
}
```

6.3.1 GET /latest_videos

Cached. Checks Supabase for videos with `video_type='latest'` cached within 6 hours. On miss, calls YouTube Search API to get video IDs, then YouTube Videos API for full metadata, caches the raw response, and returns formatted data.

YouTube API calls on cache miss: 2 (one Search, one Videos).

6.3.2 GET /homilies_videos

Not cached. Calls YouTube Playlist Items API for the Homilies playlist, then Videos API for metadata. Default 20 results.

6.3.3 GET /childrens_videos

Not cached. Playlist PLEh2brXYUf7nIigGqPthPFw4ApKiIu9zW. Same flow as homilies.

6.3.4 GET /catechism_videos

Not cached. Playlist PLEh2brXYUf7k06I4K0dgAJC11A3TlgpFK. Same flow as homilies.

6.3.5 GET /view_video_cache?video_type={type}

Admin/debug endpoint. Returns all cached videos (ignoring TTL). Optional `video_type` filter. Response:

```
{
  "count": 150,
  "videos": [
    {"video_id": "...", "title": "...", "video_type": "latest",
     "cached_at": "...", "published_at": "...", "thumbnail_url": "..."}
  ]
}
```

6.3.6 POST /cache_videos?video_type={type}&max_results={n}

Admin endpoint to force-refresh the cache. Parameters:

- `video_type`: latest, homilies, childrens, catechism, or all (default: latest)
- `max_results`: 1-50 (default: 20, capped at 50)

For each type: clears existing cache entries, fetches fresh from YouTube, stores in Supabase. Returns count per type.

6.4 Calendar Endpoint

6.4.1 GET /schedule

Returns church events for the next 10 days. Uses US/Central timezone. Events are formatted as:

```
[
  {"date": "Sun - Dec 1", "time": "9am", "service": "Divine Liturgy"},
  {"date": "", "time": "6pm", "service": "Vespers"}
]
```

The `date` field is an empty string for subsequent events on the same day (for display grouping). Times use 12-hour format with lowercase am/pm, minutes omitted when :00.

Default Calendar ID:

s37s3qvlf9nobhplqp0umlcafs@group.calendar.google.com
(overridable via `GOOGLE_CALENDAR_ID` env var).

7 Core Module Details

7.1 Ghost CMS Integration (fetch_ghost.py)

Base URL: <https://orthodox-net.ghost.io/ghost/api/content/>

Authentication: Content API key passed as `?key=` query parameter. Currently **hardcoded** in the source file (line 14).

Key functions:

- `fetch_posts()` — All posts with tags and authors.
- `fetch_tagged_posts(slug)` — Posts filtered by tag slug, single page.
- `fetch_tagged_post_data(slug)` — **Paginated** (100 per page). Returns metadata via `fetch_ids_from_tagged_post_data()`.
- `fetch_article(post_id)` — Single post; returns reformatted `{title, content, featured_image}`.
- `fetch_article_by_slug(slug)` — Uses Ghost's `filter=slug:{slug}` query; returns extended fields.
- `reformat_headings(content)` — Replaces all heading tags (`h1–h6`) with `<strong>` so the frontend controls styling.

Logging: Writes to `fetch_ghost.log` at `DEBUG` level.

7.2 YouTube Integration (`fetch_youtube.py`)

API Key: Loaded via `youtube_credentials.get_api_key()`. Currently **hardcoded** in `youtube_credentials.py`.

API Endpoints Used:

- `youtube/v3/search` — Channel search for latest videos (returns video IDs).
- `youtube/v3/playlistItems` — Playlist items (returns video IDs).
- `youtube/v3/videos` — Full video details (`snippet, statistics, contentDetails`).

Key functions:

- `fetch_latest_videos()` — Cache-aware. Checks the video cache for 'latest' type, returns cached on hit. On miss: Search API → Videos API → cache → return.
- `fetch_latest_video_ids(max_results)` — Search API call only.
- `fetch_video_data(video_ids)` — Videos API call; accepts comma-joined IDs.
- `fetch_playlist_video_ids(playlist_id, max_results)` — Playlist Items API.
- `fetch_latest_playlist_videos(playlist_id)` — Playlist IDs → video details. Used by `homilies/childrens/catechism` endpoints.
- `fetch_homilies_playlist()`, `fetch_childrens_playlist()`, `fetch_catechism_playlist()` — Convenience wrappers with hardcoded playlist IDs.

7.3 Calendar Integration (`fetch_calendar.py`)

Uses `google-api-python-client` with an API key (no OAuth). All times are processed in **US/Central** timezone.

Key functions:

- `fetch_calendar_events()` — Cache-aware main function. Calculates today + 10 days in US/Central, checks cache, falls back to API. Handles pagination. Returns raw JSON string.
- `extract_events(events_json)` — Formats raw Google Calendar events into display format (`date/time/service`). Handles all-day events, timezone conversion, date grouping (empty `date` for same-day events).
- `return_calendar_events()` — Orchestrator; calls `fetch` then `extract`.

7.4 Legacy Article Scraper (`fetch_legacy_article.py`)

Fetches HTML from `https://www.orthodox.net/articles/{filename}.html` with a browser-like User-Agent header. Extracts content between `<!--BEGIN-CONTENT-->` and `<!--END-CONTENT-->` comment markers. No caching. Logs to `fetch_orthodox_net.log`.

8 Error Handling Patterns

8.1 General Pattern

All `app.py` endpoints wrap their logic in try-except blocks and return structured JSON error responses:

- **400** — Missing required query parameters.
- **404** — No data found (empty article list, no videos, no schedule).
- **500** — Unhandled exceptions (logged, generic message returned).

8.2 Cache Failure Resilience

- If Supabase credentials are missing, caching is silently disabled at module load. The app functions normally, just without caching.
- If a cache write fails (e.g., Supabase is down), a warning is logged but the response still returns fresh data.
- If the Google Calendar API fails, `fetch_calendar.py` returns expired cached data if available, otherwise an empty array.
- If the YouTube API fails (missing `items` in response), an error message is printed and an empty list is returned.

8.3 External API Errors

- Ghost CMS errors propagate as exceptions (no explicit status code checking in `fetch_ghost.py`).
- YouTube errors are detected by checking for "items" key in response; error messages are extracted from `response.error.message`.
- Google Calendar uses the `googleapiclient` exception handling.

9 Logging

Log File	Source Module	Level
<code>app.log</code>	<code>app.py</code>	DEBUG
<code>fetch_ghost.log</code>	<code>fetch_ghost.py</code>	DEBUG
<code>fetch_orthodox.net.log</code> (console only)	<code>fetch_legacy_article.py</code>	DEBUG
(console only)	<code>fetch_calendar.py</code>	INFO
(console only)	<code>calendar_cache.py</code>	INFO
(console only)	<code>video_cache.py</code>	INFO
(console only)	<code>fetch_youtube.py</code>	(module logger)

All log files use the format: `%(asctime)s - %(levelname)s - %(message)s`.

`app.py` configures dual output: file (`app.log`) + console via a `StreamHandler`.

Cache-specific log messages to watch for:

- "USING CACHED EVENTS" / "USING CACHED VIDEOS" — Cache hit.
- "FETCHING FRESH" — Cache miss, calling external API.
- "Successfully cached N events/videos" — Cache write succeeded.
- "Failed to cache ... but continuing" — Cache write failed, non-fatal.
- "Supabase credentials not found" — Caching disabled entirely.

10 Environment Configuration

The application loads environment variables from a `.env` file via `python-dotenv`. See `.env.sample` for the complete template.

Variable	Required	Description
<code>GHOST_BASE_URL</code>	Yes*	Ghost CMS base URL. *Currently hardcoded; <code>.env.sample</code> is prepared for migration.

Variable	Required	Description
GHOST_API_KEY	Yes*	Ghost Content API key. *Currently hardcoded in <code>fetch_ghost.py</code> .
YOUTUBE_API_KEY	Yes*	YouTube Data API v3 key. *Currently hardcoded in <code>youtube_credentials.py</code> .
YOUTUBE_CHANNEL_ID	Yes*	YouTube channel ID. *Currently hardcoded in <code>fetch_youtube.py</code> .
YOUTUBE_HOMILIES_PLAYLIST	No*	Homilies playlist ID. *Hardcoded.
YOUTUBE_CHILDRENS_PLAYLIST	No*	Children's playlist ID. *Hardcoded.
YOUTUBE_CATECHISM_PLAYLIST	No*	Catechism playlist ID. *Hardcoded.
GOOGLE_API_KEY	Yes	Google API key for Calendar. Read from env at runtime.
GOOGLE_CALENDAR_ID	No	Calendar ID. Default: <code>s37s3qvlf9nobhplqp0umlcafs@...</code>
SUPABASE_URL	Yes	Supabase project URL. Read from env at runtime.
SUPABASE_KEY	Yes	Supabase service role key. Read from env at runtime.
SUPABASE_DB_URL	No	Direct PostgreSQL connection string (used by <code>db/</code> setup scripts).
RENDER	Auto	Set automatically by Render.com. Detected by <code>app.py</code> to switch Swagger to HTTPS scheme.

Critical note: Only `GOOGLE_API_KEY`, `SUPABASE_URL`, and `SUPABASE_KEY` are actually read from environment variables at runtime. Ghost CMS and YouTube credentials are **hardcoded in source files**. The `.env.sample` was created to support migrating these to env vars, but that migration has not been completed.

11 Deployment & DevOps

11.1 Hosting Platform

The application is deployed on **Render.com**. The Render project is named “**orthodox.net Flask Backend**” and contains two environments, each with a single Web Service.

Note: The Procfile in the repository originated from an earlier Heroku deployment. Render also reads this format, so it remains functional. If Render-specific features are needed in the future (e.g., health check paths, pre-deploy commands), consider adding a `render.yaml` Blueprint.

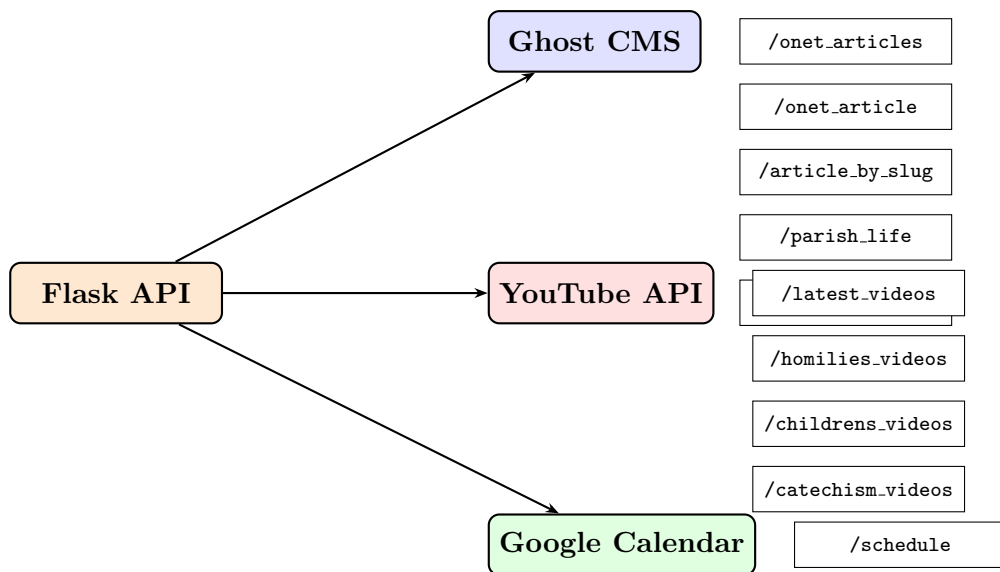


Figure 4: API endpoint groups mapped to their external data sources.

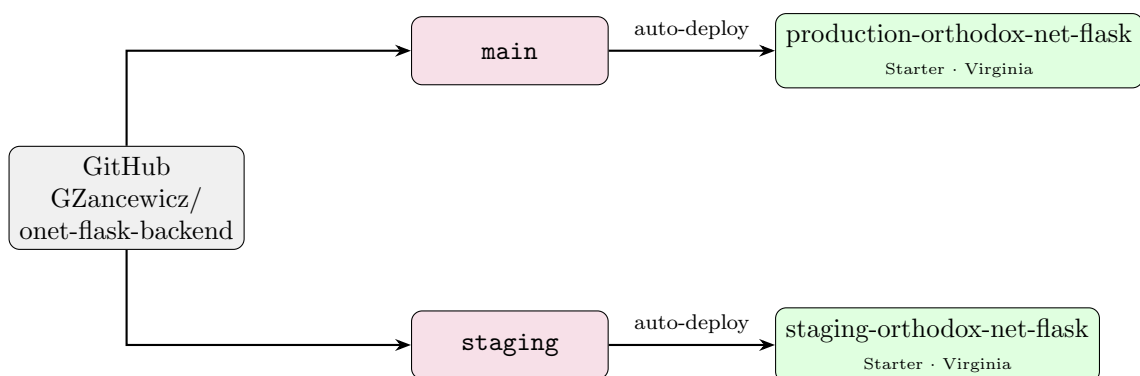


Figure 5: Deployment pipeline. Render watches each branch on GitHub and auto-deploys on push. Production: <https://production-orthodox-net-flask.onrender.com>. Staging: <https://staging-orthodox-net-flask.onrender.com>.

11.2 Render Environments

11.2.1 Production Environment

Property	Value
Render service name	production-orthodox-net-flask
Service type	Web Service
Service ID	srv-d2ce9g0gjchc7382590g
Plan	Starter
Runtime	Python 3
Region	Virginia (US East)
Git branch	main
Live URL	https://production-orthodox-net-flask.onrender.com
Swagger docs	https://production-orthodox-net-flask.onrender.com/docs
Auto-deploy	On push to main
Manual deploy	Available via Render dashboard (“Manual Deploy” button)

11.2.2 Staging Environment

Property	Value
Render service name	staging-orthodox-net-flask
Service type	Web Service
Service ID	srv-d2caeg9r0fns73dpbne0
Plan	Starter
Runtime	Python 3
Region	Virginia (US East)
Git branch	staging
Live URL	https://staging-orthodox-net-flask.onrender.com
Swagger docs	https://staging-orthodox-net-flask.onrender.com/docs
Auto-deploy	On push to staging

11.3 Render Service Configuration

Both services share the same build and runtime configuration:

Setting	Value
Build command	<code>pip install -r requirements.txt</code>
Start command	<code>gunicorn app:app</code> (from Procfile)
Python version	3.12.3 (from <code>runtime.txt</code>)

Environment variables	Configured per-service in Render dashboard
HTTPS	Provided automatically by Render
CORS	Open to all origins (<code>origins: "*"</code>)

Environment variables are configured separately for each service in the Render dashboard and are **not stored in the repository**. Both services need the same set of variables (`GOOGLE_API_KEY`, `SUPABASE_URL`, `SUPABASE_KEY`, etc.), but staging may point to different Supabase projects or use different API keys for isolation.

Render automatically sets the `RENDER` environment variable, which the application detects to switch Swagger UI to HTTPS mode.

11.4 Deployment Workflow

1. **Feature development:** Create a feature branch from `main`, make changes, open a PR against `main`.
2. **Staging test:** Merge the PR into `staging` (or push the feature branch to `staging`) to trigger a staging deploy. Verify at <https://staging-orthodox-net-flask.onrender.com>.
3. **Production release:** Merge the PR into `main`. Render auto-deploys to production.
4. **Rollback:** Use Render's "Manual Deploy" to redeploy a previous commit, or revert the Git commit and push.

11.5 Procfile

The Procfile contains: `web: gunicorn app:app`

This format originated with Heroku but Render also supports it. No Gunicorn worker count or timeout is specified, so Render/Gunicorn defaults apply. For higher traffic, consider adding `-w 4 --timeout 120`.

11.6 CORS Configuration

CORS is configured in `app.py` as:

```
1 CORS(app, resources={r"/*": {"origins": "*"}})
```

This allows **any origin** to make requests. This is intentional — the API is designed to be consumed by frontend applications running on different domains.

11.7 Swagger / HTTPS Detection

`app.py` detects the Render environment and adjusts the Swagger spec:

```

1 is_production = os.environ.get('RENDER') is not None
2 swagger_template = {
3     ...
4     "schemes": ["https"] if is_production else ["http", "https"],
5 }

```

This ensures the Swagger “Try it out” feature uses HTTPS in production.

12 Development Setup

12.1 Quick Start

1. Clone the repository:
`git clone https://github.com/GZancewicz/onet-flask-backend.git`
2. `pip install -r requirements.txt`
3. `cp .env.sample .env` and fill in credentials
(at minimum: `GOOGLE_API_KEY`, `SUPABASE_URL`, `SUPABASE_KEY`).
4. Run SQL from `db_schema.sql` and `video_cache_schema.sql` in Supabase SQL editor.
5. `python3 app.py` — server starts at `http://localhost:5001`.
6. Visit `http://localhost:5001/docs` for Swagger UI.

Note: Ghost CMS and YouTube will work without `.env` changes because their credentials are currently hardcoded. Supabase and Google Calendar require valid env vars.

12.2 Running Tests

```

pytest                # Run all tests
pytest --cov          # Run with coverage
pytest test/test_filename.py  # Run specific test file

```

12.3 Test Organization

Directory	Contents
<code>test/</code>	Main test suite: calendar fetch tests, database setup tests
<code>modules/youtube/test/</code>	YouTube module tests (unit tests with mocks + live API tests)
<code>legacy_test/ghost/</code>	Ghost CMS tests: fetch article, posts, tags, tagged posts

<code>legacy_test/youtube/</code>	Legacy YouTube tests: playlists, video data, latest videos
<code>db/test/</code>	Database setup and YouTube table creation tests

Tests use `pytest-mock` to mock external API calls. Tests with “live” in the filename make actual API calls and may fail if API keys are invalid or quotas are exhausted.

12.4 Compiling This Document

```
./compile_doc.sh      # Requires pdflatex (e.g., brew install basictex)
```

13 Security Considerations

- **Hardcoded API keys:** The Ghost CMS Content API key is hardcoded in `fetch_ghost.py` (line 14). The YouTube API key is hardcoded in `youtube_credentials.py` (line 1). Both are committed to Git. The Ghost key is read-only (low-risk by design), but the YouTube key should be migrated to an environment variable.
- **CORS wildcard:** All origins are allowed (*). This is by design but means the API is callable from any website.
- **No authentication:** The API has no authentication or rate limiting. All endpoints are public. The `POST /cache_videos` admin endpoint is also unauthenticated.
- **No input sanitization on query params:** Parameters like `article_id`, `slug`, and `filename` are passed directly to external APIs. The Ghost API and URL encoding provide some protection, but there is no explicit validation.
- **Supabase service role key:** The `SUPABASE_KEY` used is the service role key (full access), not the anon key. This is appropriate for a backend but must never be exposed to the frontend.
- **Legacy article fetching:** The `filename` parameter in `/legacy_article` is URL-encoded but not validated against a whitelist. It constructs a URL to `orthodox.net`.

14 Known Technical Debt

1. **Hardcoded credentials** — Ghost and YouTube credentials should be migrated to environment variables. The `.env.sample` is already prepared for this.
2. **Inconsistent caching** — Only `/latest_videos` uses automatic caching. The playlist endpoints (`/homilies_videos`, etc.) hit YouTube on every request and could benefit from the same pattern.

3. **Duplicate YouTube table schemas** — Both `db_schema.sql` and `video_cache_schema.sql` define a `youtube_videos` table with different columns. The simpler schema is what the code uses.
4. **Backup/dead files** — Several files appear unused: `app_backup.py`, `app_swagger_hybrid.py`, `fetch_youtube_v0.py`, and others.
5. **Dual Supabase clients** — Both cache modules independently create Supabase clients at import time. A shared client would reduce overhead.
6. **Separate logging configs** — Each module calls `logging.basicConfig()` independently, which can conflict. A centralized setup would be better.
7. **No rate limiting or auth** — The `POST /cache_videos` admin endpoint is publicly accessible with no authentication.
8. **Ghost pagination inconsistency** — `fetch_tagged_posts()` returns a single page, while `fetch_tagged_post_data()` paginates through all pages. This means `/onet_articles` may not return all articles.
9. **Procfile** — Originally for Heroku; still functional on Render but may cause confusion. Consider documenting or replacing with a `render.yaml` if Render-specific features are needed.

15 Maintenance Checklist

If taking over maintenance of this project, ensure you have the following:

1. **GitHub access:** Write access to the repository at <https://github.com/GZancewicz/onet-flask-backend>
2. **Render.com access:** Dashboard access to the “orthodox.net Flask Backend” project (both production and staging services — logs, environment variables, manual deploys)
3. **Render environment variables:** The values for `GOOGLE_API_KEY`, `SUPABASE_URL`, and `SUPABASE_KEY` configured in each Render service. These are not stored in the repository.
4. **Supabase access:** Login to the Supabase project used for caching. Required for viewing cached data, running SQL, or modifying table schemas.
5. **Ghost CMS access:** Admin login for the Ghost instance at `orthodox-net.ghost.io` for article and content management.
6. **Google API credentials:** Access to the Google Cloud project that owns the API key used for Google Calendar. Required for quota monitoring and key rotation.
7. **YouTube API credentials:** The YouTube Data API key (currently hardcoded in `youtube_credentials.py`). Access to the Google Cloud project for quota monitoring.

8. **DNS:** Understanding of who manages DNS for `orthodox.net`, in case the API needs a custom domain or subdomain in the future.
9. **Hardcoded credentials:** Be aware that Ghost and YouTube API keys are hardcoded in source files, not read from environment variables. See [Section 13](#) for details.
10. **Cache monitoring:** Familiarity with the Supabase tables (`calendar_events`, `youtube_videos`) and how to verify cache freshness. The admin endpoints `GET /view_video_cache` and `POST /cache_videos` can be used without Supabase dashboard access.

A Project History by Milestone

This appendix summarizes the development history of the orthodox.net Flask Backend organized by major milestone. The full commit history is available via `git log` on the main branch at <https://github.com/GZancewicz/onet-flask-backend>.

A.1 Phase 1: Initial Setup and Core API (Sep 2023)

The project began on September 3, 2023, bootstrapping the Flask backend with initial endpoints and deploying to Heroku.

- Created the Flask application with CORS support and a `/heartbeat` health check
- Copied utility scripts from the earlier `onet-v2` project
- Built the `/schedule` endpoint, initially parsing legacy bulletin data, then switching to the Google Calendar API
- Integrated YouTube video fetching via the YouTube Data API v3
- Configured Heroku deployment with `Procfile` and `requirements.txt`
- Resolved CORS issues for cross-origin frontend access

A.2 Phase 2: YouTube Playlists and Ghost CMS (Oct–Nov 2023)

Extended the API with curated video playlists and began integrating Ghost CMS for article management.

- Added YouTube playlist support for homilies, children’s sermons, and catechism videos
- Built the Ghost CMS Content API interface (`fetch_ghost.py`)
- Created `/test_article` and `/onet_articles` endpoints
- Added `/onet_article?article_id=` for fetching individual articles
- Implemented heading reformatting to give the frontend control over styles

A.3 Phase 3: Article Features and Metadata (Jan–Jul 2024)

Expanded article functionality with metadata endpoints and new content types.

- Added `/latest_article` endpoint
- Updated API keys and added new article endpoints

- Built article metadata extraction (title, excerpt, tags, authors)
- Developed the `/onet_article_data` endpoint for lightweight article listings

A.4 Phase 4: Testing, Database, and Infrastructure (Dec 2024)

Major infrastructure push: introduced testing, database integration, and dependency management.

- Added unit testing with `pytest` and `pytest-mock` (PR #1)
- Set up Ghost CMS unit tests
- Configured PostgreSQL database and YouTube SQL tables via SQLAlchemy
- Set up `python-dotenv` for credential management
- Pinned `Werkzeug` version to resolve Heroku deployment errors
- Built YouTube video table population (100 latest videos)
- Added test coverage reporting with `pytest-cov`
- Created playlist video retrieval from the database
- Added home page, `/onet_article_data` URL field, and feature image support
- Built legacy article scraping from `orthodox.net` (PR #5)

A.5 Phase 5: Content Endpoints and Calendar (Jan–Feb 2025)

Added parish-specific content features and improved the calendar.

- Extended calendar to fetch events 10 days out (PR #6)
- Stopped tracking log files in Git
- Added `/parish_life` endpoint for parish life articles (PR #7)
- Added `/motd` endpoint for message of the day (PR #8)
- Removed article count limits on `/onet_article_data` (PR #9)

A.6 Phase 6: Migration to Render and Supabase Caching (Aug 2025)

Major platform migration from Heroku to Render, with Supabase caching layer.

- Migrated hosting from Heroku to Render.com with staging and production environments
- Added Swagger UI documentation via Flasgger at `/docs`
- Recovered `/article_by_slug` endpoint for SEO-friendly URLs
- Moved Google Calendar credentials to environment variables
- Integrated Supabase as the caching layer for calendar events (24-hour TTL)
- Added video caching in Supabase for `/latest_videos` (6-hour TTL)
- Fixed Swagger HTTPS scheme detection for Render
- Built POST `/cache_videos` admin endpoint and `/view_video_cache`
- Created `video_cache.py` and `calendar_cache.py` modules (PR #10, #12)

A.7 Phase 7: Stabilization and Bug Fixes (Dec 2025)

Polished caching, fixed pagination bugs, and improved project hygiene.

- Enhanced caching features and updated README
- Updated `.gitignore` to ignore all `.env` files
- Fixed bug where `fetch_tagged_post_data()` was limited to one page of Ghost results, causing missing blog posts (PR #14)

B Document Revision History

Version	Date	Description
1.0	2025-05-04	Initial document creation. Basic overview of system architecture, API endpoints, core modules, logging, and deployment. Created as both <code>onet.backend.tex</code> and <code>technical_documentation.tex</code> (identical duplicates).
2.0	2026-03-15	Comprehensive rewrite. Removed duplicate <code>technical_documentation.tex</code> . Added executive summary, complete API reference with request/response examples, full database schemas (SQL), caching layer details with TTL and fallback behavior, module dependency graph, Supabase client initialization patterns, error handling patterns, environment variable reference table, security considerations, known technical debt (9 items). Added Render.com deployment details with service IDs, URLs, and deployment workflow. Added TikZ diagrams (system architecture, module dependencies, cache flow, endpoint map, deployment pipeline). Added project history appendix (7 phases). Renamed “Orthodox Net” to “orthodox.net” throughout. Moved files to <code>doc/latex/onet_backend/</code> .